

Operating systems

Lecture #2

Content Lecture#2

01	Interrupts – hardware, software, and exception
02	Process
03	Concept of Process
04	Operations on Process
05	Process creation & termination
06	Process Control Block
07	Process Table
08	Process State &Process State Transition
09	Process Context Switch
10	Zombie and Orphan Process

Lets take a Break



- where Saint Peter gives him a nice, modern six-bedroom house with a pretty garden and a tennis court. Pleased with his lot, Bill quickly settles into the afterlife.

  One day he is out walking when he bumps into a man wearing a fine tailored suit.

  "That's really nice," says Bill. "Where did you get it?"

  "Actually," says the man, "I was given 50 of these, plus two mansions, a yacht, a golf course and four Rolls-Royces."

  "Wow, were you a pope or a doctor healing the terminally ill?" asks Bill.

  "No, I was the captain of the Titanic."

  Bill storms off to see Saint Peter. "How come the captain of a sunken ship gets all that while I, the inventor of the Windows Operating System gets a crummy little house?" he asks.

  Saint Peter replies, "The Titanic only crashed once."



Interrupts

An interrupt is **a signal emitted by a device attached to a computer or from a program within the computer**. It requires the operating system (OS) to stop and figure out what to do next.

Process

A process is **a program in execution.**

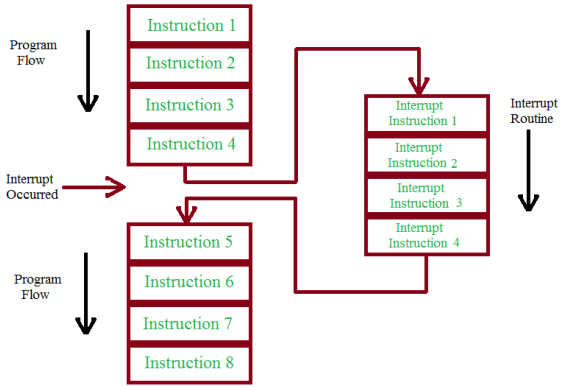
Interrupts

- Mechanism by which Programs/hardware like I/O devices, memory modules may interrupt the normal processing of the processor
- Interrupts are generated by various agents to notify the OS of the occurrence of some event
  - such as completion of an I/O activity
- When an interrupt occurs, the execution flow of processor is diverted into specific part of OS code which deals with the event – *interrupt handler*
- Interrupts are used to permit several programs and I/O activities to process independently and asynchronously
  - Hence improve the processing speed

- Advantages
  - It provides a low overheads means of gaining the attention of the CPU
  - *Interrupt Vector Table* is a location near the bottom of memory contains the address of interrupt services procedures of I/O devices
- As each instruction terminates, the processor may checks for the occurrence of interrupts
  - If an interrupt received, the actual program is temporarily suspended, and the processor is diverted to interrupt handling routine
  - When the interrupt is served, the execution will return to the interrupted program

Interrupt Processing

- Following events occurs in processor hardware and software due to an I/O interrupt
  - The device issue an interrupt signal to processor
  - The processor finishes execution of current instruction, saves states of interrupted process before responding to the interrupt
  - The processor sends an acknowledgment signal to the device that issued the interrupt and device remove its interrupt signal/flag
  - Processor transfer control to the appropriate interrupt handler routine and it starts processing interrupt
  - When the interrupt id served, the state of interrupted process is restored
  - Next process starts execution



Interrupt Service Routine

For every interrupt, there must be an interrupt service routine (ISR), or **interrupt handler**. When an interrupt occurs, the microcontroller runs the interrupt service routine. For every interrupt, there is a fixed location in memory that holds the address of its interrupt service routine, ISR. The table of memory locations set aside to hold the addresses of ISRs is called as the Interrupt Vector Table.

Types of Interrupts

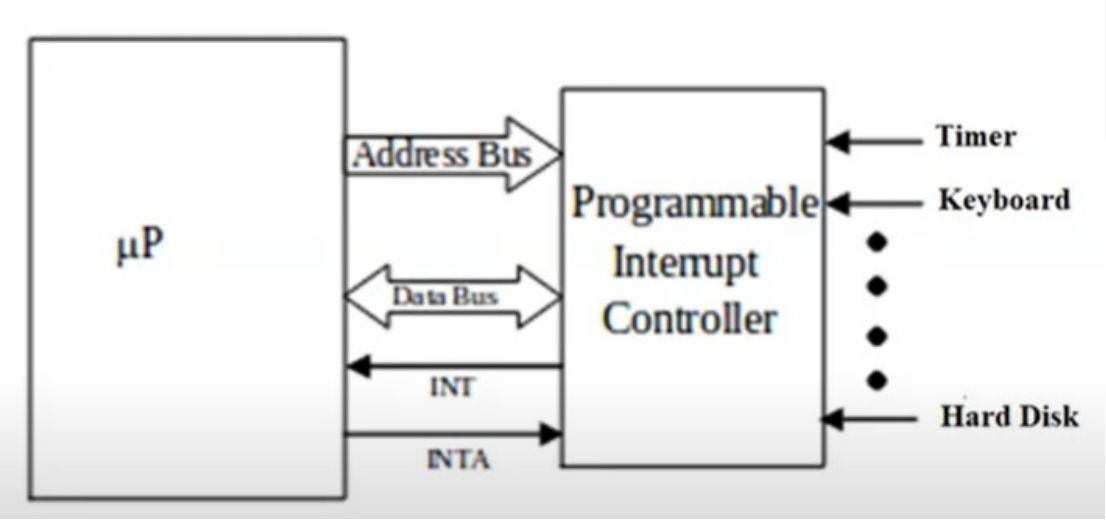
Interrupts are categorized in following

- Hardware Interrupts
- Software interrupts
- Exception

Interrupts may be generated by a number of sources, which are following

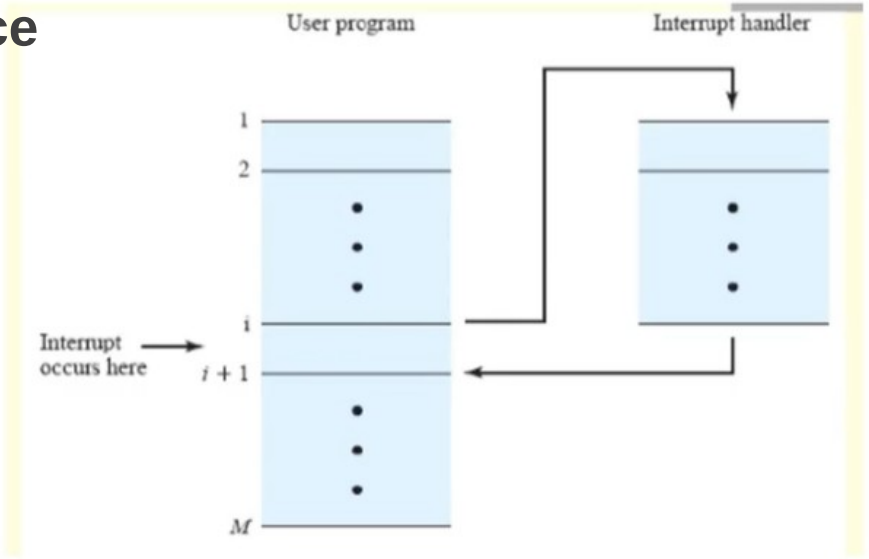
- **I/O:** Generated by I/O device controller, to signal normal completion of event or the occurrence of an error or failure condition
- **Time:** Generated by an internal clock within the processor due to expiration of a process's time quantum or receipt of a signal from another processor on a multiprocessor system
- **Hardware error:** Generated by hardware faults such as power failure
- Exception
    - Trap: control returns to the next instruction which caused interrupts
    - Fault: Control returns to the same instruction which caused interrupts
    - Double Fault Exception within Exception
- **Program:** Generated in a user program as a result of instruction execution, such as;
    - Arithmetic overflow
    - Divide by zero
    - Attempt to execute an illegal machine instruction
    - Reference out side a user allowed memory space

How Hardware interrupts are generated



How Software interrupts are generated

Application Programming Interface (API)



System Call &API

Application Programming Interface (API)

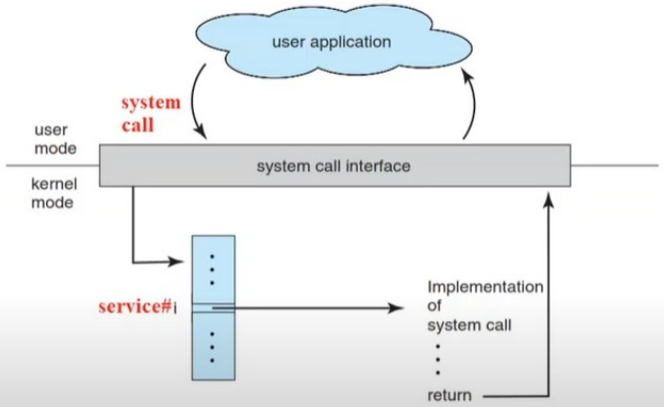
```
Print (char* str)
{
DX ← str
Ah ← 0x9
int  0x21

return
}
```

DX<- Data for Kernel
Ah <- Service#
Int #num

System Call

DX ← Data for kernel
Ah ← Service#
int #system call num



Process

- A process is basically a program in execution
  - or that which a processor executes
- A process is unit of work in a system
- A process is an activity performed by a system
  - It may be program’s data, program counter, pointer or register used in a program
- A process uses resources like CPU, memory, files and I/O devices
- Operating system processes execute system code and user processes executes user code

Program VS Process

- A program is a passive entity
- A process is a dynamic entity represented by the values stored in registers, memory, program counters etc.
- A program does not compete for the computing resources whereas a process does
- One program can have several processes
    - Consider multiple users executing the same program

Process Concept

- Process idea was first used in Multics
- All programming OS, such as Windows NT, or MVS (multiple virtual storage) were built around the concept of process
- Major requirements that the OS must meet can all be expressed with reference to process
  - OS must interleave the execution of a number of processes to maximize processor use
  - OS must allocate resources to process in confirming to a specify policy
    - e.g. certain functions or applications are of higher priority
    - at the same time avoiding deadlock
  - OS may support inter-process communication

Program

- Structure of Binary file

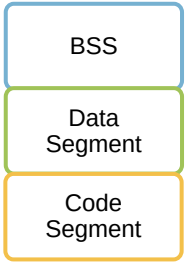
  - Executable instruction
  - Globally initialized variable
  - Uninitialized Global variable
  - Local initialized variable
  - Local Uninitialized variable

C++ Program

```
#include <iostream>
using namespace std;

int a=10;
int b=5;

  main()
      {
   // Write C++ code here
int c;
c=a+b;
cout << "Hello world!\n\n";
cout << c;    return 0;
    }
```



- **Globally initialized** variable are kept in data segment
- **Uninitialized Global** variable are kept in BSS
- ***Local*** variables are kept in code segment as part of

Code Segment
One of the sections of a program in an object file or in memory, which contains executable instructions.

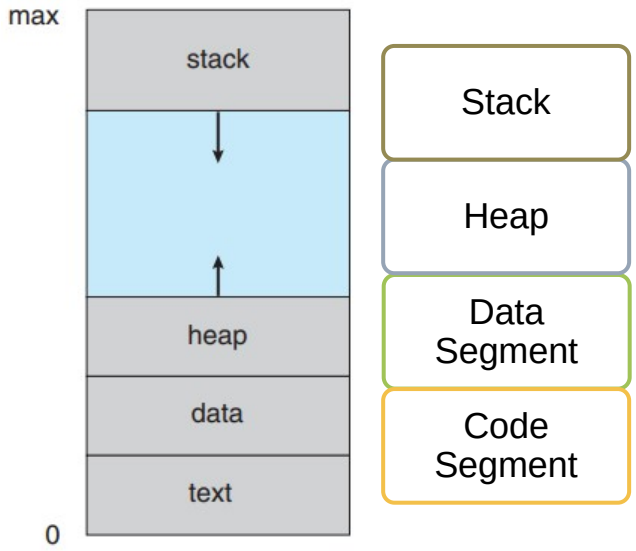
Data Segment
A data segment is a portion of the virtual address space of a program, which contains the global variables and static variables that are initialized by the programmer.

BSS(block started by symbol)
Data in this segment is initialized by the kernel to arithmetic 0 before the program starts executing
uninitialized data starts at the end of the data segment and contains all global variables and static variables that are initialized to zero or do not have explicit initialization in source code..

• Process

Heap
is memory that is dynamically allocated during process run time. The heap segment contains dynamically allocated memory, commonly begins at the end of the BSS segment and grows to larger addresses from there. It is managed by malloc, calloc, realloc, and free, which may use the brk and sbrk system calls to adjust its size. The heap segment is shared by all threads, shared libraries, and dynamically loaded modules in a process.

Stack
A process generally includes the process stack, which contains temporary data (such as function parameters, return addresses, and local variables), The stack area traditionally adjoined the heap area and grew in the opposite direction. The stack area contains the program stack, a LIFO structure, typically located in the higher parts of memory. The set of values pushed for one function call is termed a “stack frame”



Local uninitialized or uninitialized **variables are kept in stack until function returns**

Activity

You have an array of 1000 elements and you need to search a number from it.

2 core available or 4 core available
Parallel processing can be achieved if a programmer speed up the performance by assigning a part of program to each core

For a single processor
Multitasking
Divide single program in multiple process and assign it to more time slice and time is assigned more to a single process

Operation on process

- A system managing processes may perform following operations
  - Create a process
  - Destroy a process
  - Suspend a process
  - Resume a process
  - Block a process
  - Wakeup a process
  - Dispatch a process
  - Change process’s priority
  - Enable a process to communicate with other processes
- For long term suspension, process’s resources should be free
- Changing the priority of process normally involved modifying the priority value in PCB

Process creation & termination

- A process may create other processes
- When a process creates a new process
  - The creator process is called the parent process
  - Created process is called the child process
- Act of creating a process is called *spawning* a process
- A child process may share the resources of its parent process
- A parent may wait for its child process to terminate before resuming or may run concurrently with the child process
- A child process may run the same program as that of its parent or may run a separate program
- In DOS, parent and child process do not run in parallel and parent process will suspend itself till child has finished

Process creation & termination

- When a process is created the following operations are performed
  - Process naming (process id)
  - Creates the process control block
  - Insert it in process table
  - Determine the process initial priority
  - Allocate the process's initial resources
  - Allocate the address space to be used by the process
- When a process is destroyed following operations are performed
  - Its resources are returned to the system
  - Its process control block is erased
  - It is purged from any system list or table

Reason: Process creation & termination

Reason of Creation

1. New batch job
   - In a batch environment, a process is created in response to the submission of a job
2. Interactive log on
   - A user at a terminal logs on to the system
3. Created by OS to provide a service
   - OS can create a process to perform a function on behalf of a user program, without user having to wait, e.g. printing
4. Spawned by existing process
   - A user program can create a number of processes
   - Typically related processes need to communicate and cooperate each other
     - Achieving this cooperation is a difficult task for the programmer of OS

Reason: Process creation & termination

Reason of termination

There must be some means for a process to indicate its completion

A batch job should include a halt instruction, which generates an interrupts to alert the OS that a process has completed

For an interactive application, the action of the user will indicate whether the process is completed

• Reason: Process creation & termination

Following are the possible reasons for process termination;

1. **Normal termination :**The process executes an OS service call to indicate that it has completed running
2. **Time limit exceeds**: The process has run longer than the specified total time limit which includes

total time elapsed , amount of time spent executing ,

in case of interactive process, he amount of time since the user last provided any input

3. **Memory unavailable**: The process requires more memory than the system can provide
4. **Bounds violation**: The process tries to access the memory locations that is not allowed to access
4. **Protection error:** The process attempt to use a file that is not allowed to use or, It tries to use it in improper fashion, such as writing to a read only file
4. **Arithmetic error:** The process tries a prohibited computation such as division by zero
4. **Time overrun:** The process has waited longer than specified maximum for a certain events to occur

Reason: Process creation & termination

8. **I/O failure**: An error occur in input or output, such as
inability to find a file
failure to read a write after a specified maximum no of tries (when, e.g. a defective area is encountered on a diskette) or
invalid operation (such as reading from the line printer)
8. **Invalid instruction:** The process attempts to execute a non-existent instruction (often a result of branching into a data area and attempting to execute data)
8. **Privileged instruction:** The process attempts to use an instruction reserved for the operating system
8. **Data misuse:** A piece of data is of the wrong type or is not initialized
8. **OS or operator intervention**: For some reason, the operator or the OS has terminated the process (e.g. if a deadlock exists)
8. **Parent termination:** When a parent terminates, the OS should be design to automatically terminate all the offspring of that parent
8. **Parent request:** A parent process typically has the authority to terminate any of its offspring

Process Control Block

- Each process is represented in the operating system by a process control block (PCB)—also called a task control block.
- When a new user program is initiated
  - OS creates a data structure, called Process Control Block (PCB), which gives substance to the process and serves to control it
- Each process is represented by its own process control block
- PCB is a data block or record containing information associated with a specified process
- Allows the OS to locate all key information about a process
- When the OS switches the attention of the CPU among various active processes
  - It uses the same area in PCB to hold the information
  - That information used to restart process when the process next gets CPU
- Process control block is also called task control block

Process Control Block

- PCB contains following information about process
- Current state of the process
- Unique identification of process
- A pointer to process parent
- Pointer to process's child processes
- The process's priority
- Pointer to locate the process memory
- Pointer to allocate resources
- The register save area – CPU register
- The processor it is running on
- Accounting information
- I/O state information

Pointer	Process state
Process number	
Program counter	
Registers	
Memory limits	
List of open files	
. . .	

Process Table

- OS maintains the process table with one entry per process
- This entry contains information about process’s
  - state
  - its program counter
  - stack pointer
  - memory allocation
  - the status of its open file
  - its accounting and scheduling information
  - its priority

Process State

As a process executes, it changes state. The state of a process is defined in part by the current activity of that process. A process may be in one of the following states:

- New. The process is being created.
- Running. Instructions are being executed.
- Waiting. The process is waiting for some event to occur (such as an I/O completion or reception of a signal).
- Ready. The process is waiting to be assigned to a processor.
- Terminated. The process has finished execution.

Process State

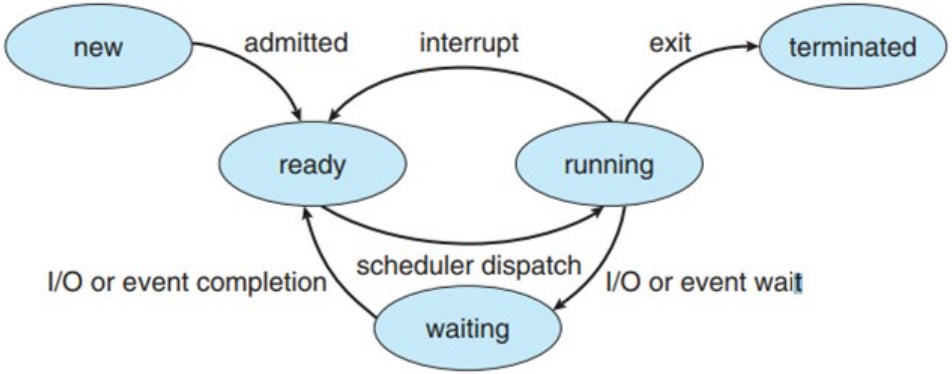


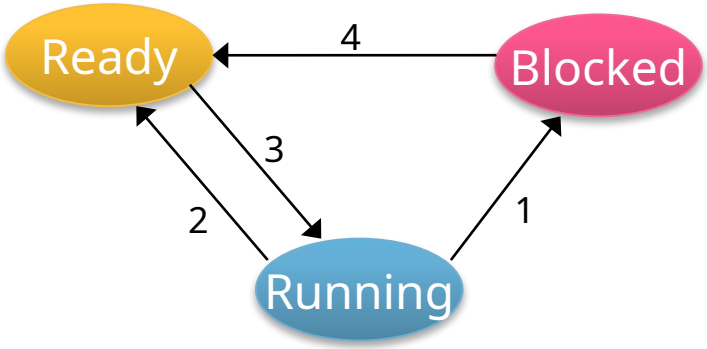
Diagram of process state.

These names are arbitrary, and they vary across operating systems. The states that they represent are found on all systems, however. Certain operating systems also more finely delineate process states. It is important to realize that only one process can be running on any processor at any instant. Many processes may be ready and waiting, however. The state diagram corresponding to these states is presented in Figure

Process State Transitions

- Transition-1:
  - Occurs when a process cannot continue may be due to I/O request
- Transition-2:
  - Request when scheduler decides that the running process has run long enough , and it’s the time to give CPU to other processes in ready state
- Transition-3:
  - Occurs when all the other process have had their CPU time and its time for the first process to run again
- Transition-4:
  - Occurs when external event for which process was waiting happens. If no other process is running at that time, transition-3 will be triggered and the process will start execution

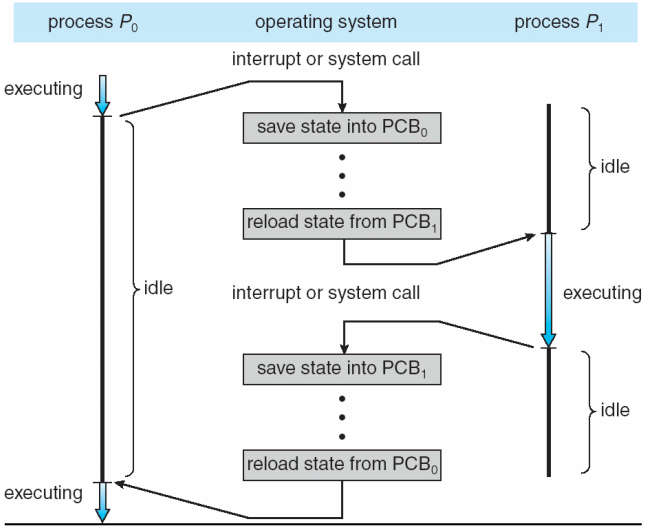
Process State Transitions



- The four transitions can be indicated as follows
  - Block running → block
  - Timerunout running → ready
  - Dispatch ready → running
  - Wakeup block → ready

Context Switch

- Context switching is the procedure of switching of process from processor to memory and vice versa in a multiprogramming environment
- When CPU switches to another process
  - The system must save the state of the old process
  - Load the saved state of the new process
- Context switch involves storing and loading the values of each of the process's control block, and process registers, such as;
  - The program counter
  - Index register
  - General register
  - Stack pointer etc



- Context switch is pure overhead
  - The system does no useful work while switching
- Context switch time depends on hardware support e.g, memory speed, the number of registers etc.
- Its range is from 1 to 100 microsecond
  - This speed vary from system to system

Zombie or Orphan Process

Immediate Parent process terminates before child process the process is called orphan process and it is adopted by the init process.

Child process terminates before wait call of parent process is called Zombie Process.
Child process terminated but id remains in process table till the wait call.

And when zombie terminates before wait call of parent in this case init process terminate zombie

Process table contain entry of the process but the process is terminated this is Zombie

• Create Process

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()

{
printf(" Hello\n");
    fork();
 printf("world\n");
 }
```

Negative Value: creation of a child process was unsuccessful.
Zero: Returned to the newly created child process.
Positive value: Returned to parent or caller. The value contains process ID of newly created child process.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
pid_t pid;          /* fork a child process */
pid = fork();
if (pid < 0)
 { /* error occurred */
fprintf(stderr, "Fork Failed");
return 1;
}
else if (pid == 0) { /* child process */
execlp("/bin/ls","ls",NULL);
}else { /* parent process *//* parent will wait for the
child to complete */wait(NULL)
;printf("Child Complete");
}return 0;
}
```

Create Process

Parent: *got its own definition with v=5*
Child: *got its own definition with v=5*

Sot the output will be 4 and 6

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
int v=5;
if(fork()==0)
{
        v++;
        printf("Value of V in chilf is %d \n",v);
        return 0;
}
else
{
        v--;
        printf("Value of V is Parent is %d \n",v);
        return 0;
}
}
```

Create Process

malloc: reserve memory on heap

```
#include<stdio.h>
#include<sys/wait.h>
#include<unistd.h>
#include<stdlib.h>

int main()
{

int *v =(int*)malloc(sizeof(int));
*v=100;
                if(fork()==1)
                {
                (*v)++;
                printf("value of v is the child process is %d \n",*v);
                return 0;
                }
                else
                {
                (*v)--;
                printf("value of v is the child process is %d \n",*v);
                return 0;
                }
}
```

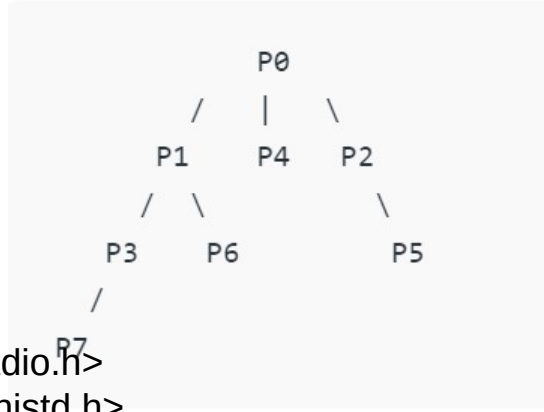
Create Process

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
  pid_t pid = fork();

  if(pid == 0) {
    printf("Child => PPID: %d PID: %d\n", getppid(), getpid());
    exit(EXIT_SUCCESS);
  }
  else if(pid > 0) {
    printf("Parent => PID: %d\n", getpid());
    printf("Waiting for child process to finish.\n");
    wait(NULL);
    printf("Child process finished.\n");
  }
  else {
    printf("Unable to create child process.\n");
  }

  return EXIT_SUCCESS;
}
```




```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()

{
printf(" Hello\n");
    pid_t pid = fork();
    printf("World",getppid(), getpid());
 fork();
 printf("Child => PPID: %d PID:
%d\n", getppid(), getpid());

 }
```

Create Process

A call to wait() blocks the calling process until one of its child processes exits or a signal is received. After child process terminates, parent ***continues*** its execution after wait system call instruction.
Child process may terminate due to any of these:
•It calls exit();
•It returns (an int) from main
•It receives a signal (from the OS or another process) whose default action is to terminate.

Lab

- Run a program for adding 2 numbers and define the concept of Program and Process
- Create a Process with Fork()



Thank you